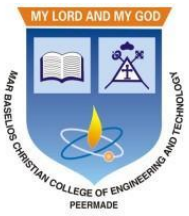
DESIGN AND SIMULATION OF 4-BIT ARITHMETIC LOGIC UNIT (ALU)

A Micro Project Report

Submitted to the APJ Abdul Kalam Technological University
in partial fulfillment for the award of the Degree of
Bachelor of Technology
in
ELECTRONICS AND COMMUNICATION ENGINEERING

by

EDWIN JOSEPH	**MBC25EC005**
JAYADEV N.S	**MBC25EC006**
ALEX BENNY	**MBC25EC002**



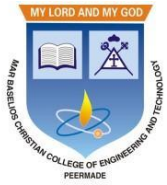
DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING
MAR BASELIOS CHRISTIAN COLLEGE OF ENGINEERING AND TECHNOLOGY
Kuttikkanam, 685531

October 2026

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

MAR BASELIOS CHRISTIAN COLLEGE OF ENGINEERING AND TECHNOLOGY

Kuttikkanam, 685531



CERTIFICATE

This is to certify that the Micro Project report entitled **Design and Simulation of 4-Bit Arithmetic Logic Unit (ALU)** is bonafide record of the work done by **Edwin Joseph (MBC25EC005), Jayadev N.S (MBC25EC006), Alex Benny (MBC25EC002)** under our guidance towards the partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology** in **Electronics and Communication Engineering** of **APJ Abdul Kalam Technological University** during the year 2026 and this work has not been submitted elsewhere for the award of any degree.

Prof. Nimisha George
Assistant Professor
Project Guide
Department of ECE

Prof. Elias Janson K
Associate Professor
Head of Department
Department of ECE

ACKNOWLEDGMENT

We would like to express our heartfelt gratitude to everyone who contributed to the successful completion of this micro-project. It would not have been possible without the timely help and guidance rendered by many individuals, to whom we feel deeply obliged and grateful. First and foremost, we extend our sincere thanks to **Dr. Santhosh Simon**, our Principal, for his constant support and encouragement. We also express our gratitude to **Prof. Elias Janson K**, Head of the Department, for providing us with the necessary resources and a conducive learning environment to carry out this project. We acknowledge from the heart to our guide, **Prof. Nimisha George**, for her valuable feedback and guidance to carry out this micro-project. Her feedback and criticism helped us in our progress and overcoming obstacles. We appreciate her for the time she gave us as our mentor; otherwise, we wouldn't have been able to complete this project.

About all, we would like to express our sincere thanks to almighty god who empowered us to fulfill the work, by showering his abundant grace and mercy.

EDWIN JOSEPH
JAYADEV N.S
ALEX BENNY

ABSTRACT

This micro-project presents the design and simulation of a 4-bit Arithmetic Logic Unit (ALU) using digital logic circuits. The proposed ALU is designed to perform a comprehensive set of arithmetic and logical operations on two 4-bit binary input vectors under the control of a 3-bit operation selector. The arithmetic section performs binary addition and 2's complement subtraction, while the logic section performs bitwise NAND, NOR, XOR, AND, OR, and NOT operations. The required operation is dynamically selected using an 8-to-1 multiplexer routing architecture, establishing a robust, modular, and scalable datapath.

The circuit is synthesized and functionally verified using Logisim Evolution, an open-source digital logic design and simulation platform. The project demonstrates core principles of digital computer engineering, including Boolean algebra, universal logic decomposition, ripple-carry addition, subtractor design via 2's complement arithmetic, bus steering through multiplexers, and interactive hexadecimal input/output visualization. Exhaustive functional simulations across all operational modes substantiate the arithmetic precision and transient stability of the design.

The completed 4-bit ALU functions as a concrete physical model of the execution datapath within modern microprocessor architectures. By structuring the architecture hierarchically into dedicated arithmetic and logic subcircuits, the design provides clear pedagogical insights into hardware reuse, critical path propagation latency, and combinational status flag generation.

CONTENTS

ACKNOWLEDGMENT i

ABSTRACT ii

LIST OF FIGURES v

LIST OF TABLES v

Chapter 1. INTRODUCTION 1

1.1 Background of Arithmetic Logic Units . . 1

1.2 Motivation and Scope 1

1.3 Objectives of the 4-Bit ALU Microproject . . 2

1.4 ALU Specification & Operations Summary . . 2

1.5 Architectural Design Constraints & Standards . . 3

1.6 Organisation of Report 3

Chapter 2. LITERATURE SURVEY & THEORETICAL FOUNDATIONS 4

2.1 Evolution of Digital ALUs in Processor Architectures . . 4

2.2 Binary Arithmetic Principles: 2's Complement Logic . . 4

2.3 Multiplexer-Based Datapath Control & Routing . . 5

2.4 Logic Design Simulation using Logisim Evolution . . 5

2.5 Comparative Study of Arithmetic Adder Topologies . . 5

Chapter 3. SYSTEM ARCHITECTURE & TOP-LEVEL DESIGN 6

3.1 Top-Level Architecture Overview . . . 6

3.2 Sub-System Decomposition & Interconnects . . 6

3.2.1 Input Operands & Interactive Hex Displays . . 6

3.2.2 8-to-1 Multiplexer Selection Stage . . 7

3.2.3 Output Interface & Visual Indicators . . 7

Chapter 4. DETAILED GATE-LEVEL CIRCUIT IMPLEMENTATION 8

4.1 Arithmetic Unit: 4-Bit Full Adder & Subtractor . . 8

4.2 Logic Unit: NAND, NOR, XOR, AND, OR, NOT Implementation . . 9

4.3 Carry Flag Logic & Multiplexed Status Output . . 10

Chapter 5. SIMULATION & FUNCTIONAL VERIFICATION **11**
5.1 Simulation Environment: Logisim Evolution . . 11
5.2 Testbench Methodology & Vector Application . . 11
5.3 Simulation Observations & Transient Behavior . . 12
Chapter 6. WORKING PRINCIPLE & OPERATION MODES **13**
6.1 Detailed Opcode Walkthrough (000 to 111) . . 13
6.2 Propagation Delay & Critical Path in Ripple Adders . . 14
Chapter 7. RESULTS & DISCUSSION **15**
7.1 Simulation Verification Results . . . 15
7.2 In-Depth Analysis of Simulation Results . . 15
Chapter 8. ADVANTAGES, APPLICATIONS & FUTURE SCOPE **17**
8.1 Advantages of Modular Gate-Level ALU Design . . 17
8.2 Practical Applications 17
8.3 Limitations & Future Scope 17
Chapter 9. CONCLUSION **19**
REFERENCES **20**
APPENDIX: LOGISIM CIRCUIT PIN & SIGNAL MAPPING **21**

LIST OF FIGURES

3.1	Top-Level 4-Bit ALU System Schematic in Logisim Evolution . .	**6**
4.1	Gate-Level Schematic of 4-Bit Binary Ripple Carry Full Adder Circuit . .	**9**
4.2	4-Bit Logic Operations Subcircuit in Logisim Evolution . .	**10**
4.3	Carry and Status Flag Multiplexer Routing Subcircuit in Logisim Evolution . .	**10**
5.1	Functional Simulation of 4-Bit ALU in Logisim Evolution under Dynamic Operands . .	**11**

LIST OF TABLES

1.1	ALU Function Select & Operation Encoding Summary (3-bit Selector) . .	**3**
4.1	Truth Table of 1-Bit Full Adder Stage . . .	**8**
7.1	Simulation Test Vector Verification Results (Arithmetic & Logic Tests) . .	**15**
A.1	Logisim Circuit Bus & Pin Connection Mapping Table . .	**21**

Chapter 1
INTRODUCTION

1.1 BACKGROUND OF ARITHMETIC LOGIC UNITS

The Arithmetic Logic Unit (ALU) is the foundational computational core of every central processing unit (CPU), microcontroller, and digital signal processor (DSP) [1], [3]. Modern microprocessors execute billions of instructions per second, yet nearly every instruction—from mathematical calculations and array indexing to conditional branching and memory address generation—relies fundamentally on the elementary operations executed within the ALU [3].

At its core, an ALU is a multi-function combinational logic circuit capable of performing a predetermined set of arithmetic operations (such as addition and subtraction) and bitwise logical operations (such as AND, OR, XOR, NAND, NOR, and NOT) on two binary input operands. The desired operation is determined at run-time by applying an operation select code (opcode) to control lines connected to internal multiplexers [1], [4]. By centralizing arithmetic and logical processing within a single modular datapath, computer architectures achieve exceptional efficiency, scalability, and programmability.

1.2 MOTIVATION AND SCOPE

In academic study and computer engineering practice, designing an ALU from discrete gate-level primitives provides invaluable insight into the internal workings of digital computers [1], [2]. While commercial microprocessors employ complex 64-bit pipelined ALUs with carry-lookahead acceleration, the underlying principles of binary addition, 2's complement subtraction, Boolean algebra, and multiplexer routing are identical to those of a 4-bit ALU [3].

This microproject focuses on the comprehensive design, hierarchical modeling, and functional simulation of a 4-bit ALU using Logisim Evolution [5]. The project bridges the gap between theoretical combinational logic design and practical datapath implementation, providing a tangible model of how elementary gates combine into intelligent processing blocks.

1.3 OBJECTIVES OF THE 4-BIT ALU MICROPROJECT

The primary technical objectives of this project include:

- **Full Adder Subcircuit:** Designing a 4-bit binary ripple adder constructed from four cascaded 1-bit full adder cells built strictly using basic XOR, AND, and OR gates [1], [2].
- **Subtractor Logic:** Implementing a 4-bit binary subtractor utilizing 2's complement arithmetic principles by complementing operand B and injecting a carry-in of 1 [2].
- **Parallel Logic Unit:** Designing a dedicated 4-bit logic processing unit capable of concurrent bitwise NAND, NOR, XOR, AND, OR, and NOT operations [4].
- **Multiplexer Routing:** Implementing an 8-to-1 bus multiplexer controlled by a 3-bit selection bus (Sin[2..0]) to route the desired computation to master output Y[3..0] [3], [4].
- **Dedicated Carry Flag:** Developing carry routing logic to forward arithmetic carry and borrow flags to output Cout while isolating it during logic operations [1], [6].
- **Hexadecimal Visualization:** Interfacing input/output hexadecimal 7-segment displays for instantaneous visual operand and result verification in Logisim Evolution [5].
- **Exhaustive Verification:** Validating the circuit across all 8 operation modes in Logisim Evolution under exhaustive test vector conditions [5].

1.4 ALU SPECIFICATION & OPERATIONS SUMMARY

The proposed 4-bit ALU processes two 4-bit binary input vectors, A = (A3, A2, A1, A0) and B = (B3, B2, B1, B0), under the control of a 3-bit selection vector, S = (S2, S1, S0), conforming to classical function generator standards exemplified by the SN74LS181 [6]. The operations are encoded as summarized in Table 1.1:

S2	S1	S0	Operation Selected	Output Expression (Y)
0	0	0	Addition (A + B)	Y = A + B (with Cout)
0	0	1	Subtraction (A - B)	Y = A - B (with Borrow/Cout)
0	1	0	Bitwise NAND	Y = ~(A & B)
0	1	1	Bitwise NOR	Y = ~(A | B)
1	0	0	Bitwise XOR	Y = A ^ B
1	0	1	Bitwise AND	Y = A & B
1	1	0	Bitwise OR	Y = A | B
1	1	1	Bitwise NOT	Y = ~A

Table 1.1: ALU Function Select & Operation Encoding Summary (3-bit Selector)

1.5 ARCHITECTURAL DESIGN CONSTRAINTS & STANDARDS

To preserve academic rigor and ensure realistic hardware behavior, the following structural constraints were applied during design:

- **Discrete Gate Primitives:** All computational sub-circuits (Add, Sub, Logic) are synthesized strictly from elementary discrete gates (AND, OR, NOT, XOR, NAND, NOR) rather than pre-packaged behavioral arithmetic blocks [1].
- **Modular Bus Topology:** All multi-bit data paths are encapsulated within 4-bit structured buses to eliminate routing clutter and maintain schematic legibility [5].
- **Synchronous Visibility:** Every operand input and intermediate output bus is routed to interactive hex displays and binary probes to allow immediate visual verification during simulation stepping [5].

1.6 ORGANISATION OF REPORT

The remaining chapters of this report are organized as follows: Chapter 2 discusses literature survey and foundational theory covering binary adders, 2's complement logic, multiplexer steering, and Logisim Evolution simulation. Chapter 3 presents the top-level system architecture and sub-system decomposition. Chapter 4 provides detailed gate-level

circuit designs for the arithmetic, logic, and status flag subcircuits. Chapter 5 details simulation methodologies and test vector validation. Chapter 6 provides an exhaustive opcode-by-opcode walkthrough and propagation delay analysis. Chapter 7 analyzes simulation results and waveforms. Chapter 8 explores design advantages, practical applications, and future enhancements. Chapter 9 concludes the report, followed by references and a comprehensive signal pinout appendix.

Chapter 2
LITERATURE SURVEY & THEORETICAL FOUNDATIONS

2.1 EVOLUTION OF DIGITAL ALUS IN PROCESSOR ARCHITECTURES

The conceptual origin of the Arithmetic Logic Unit dates back to 1945, when John von Neumann introduced the concept of a centralized arithmetic organ in his foundational EDVAC architecture report [1], [3]. In early vacuum tube computers, arithmetic execution was distributed across disparate sequential accumulators. The emergence of solid-state transistor-transistor logic (TTL) in the late 1960s enabled the consolidation of arithmetic and logic into single dedicated IC packages [2].

In 1970, Texas Instruments introduced the SN74181 [6], the industry's first single-chip 4-bit TTL ALU slice. The 74181 could perform 16 arithmetic operations and 16 logic operations on two 4-bit words, controlled by a 4-bit function select bus and an arithmetic/logic mode line. It became the computational backbone of classic minicomputers such as the DEC PDP-11 and Data General Nova [3]. Modern ALUs have scaled from 4-bit nibble slices to 64-bit wide vector processing units, incorporating speculative carry-lookahead trees and SIMD extensions [1].

2.2 BINARY ARITHMETIC PRINCIPLES: 2'S COMPLEMENT LOGIC

In digital hardware, binary subtraction is conventionally performed by adding the 2's complement of the subtrahend to the minuend, as articulated in classic digital logic texts by Tocci et al. [2] and Mano and Ciletti [1]. For an n-bit binary integer B, its 2's complement is defined mathematically as:

2's Complement of B = ~B + 1

Therefore, the arithmetic subtraction Y = A - B is computed as Y = A + (~B) + 1. This mathematical equivalence enables hardware designers to reuse an existing binary adder circuit for subtraction: each bit of operand B is inverted through an inverter or XOR gate, and a carry-in (Cin) of 1 is injected into the least significant bit (LSB) stage [1], [2]. This achieves dual-function addition/subtraction with minimal additional gate overhead.

2.3 MULTIPLEXER-BASED DATAPATH CONTROL & ROUTING

A multiplexer (MUX) is a combinational digital selector that channels one of several input data streams into a single output line based on binary selection signals [3], [4]. In processor datapaths, multiplexers act as the central routing fabric, directing computed results from distinct operational units into common register files or memory buses [3]. For an 8-to-1 multiplexer operating on 4-bit data buses, the circuit takes eight 4-bit inputs (D0 through D7) and selects exactly one bus according to 3-bit select vector Sin = (S2, S1, S0) [4].

2.4 LOGIC DESIGN SIMULATION USING LOGISIM EVOLUTION

Logisim Evolution is an open-source educational digital logic design and simulation environment widely utilized in computer engineering curricula [5]. It provides real-time event-driven simulation of combinational and sequential logic, dynamic propagation delay modeling, bus bundle management, and subcircuit encapsulation. Unlike command-line HDL simulators, Logisim Evolution provides instant visual feedback through interactive probe displays, animated wire states, and 7-segment hex decoders, making it ideal for verifying gate-level architectures [5].

2.5 COMPARATIVE STUDY OF ARITHMETIC ADDER TOPOLOGIES

A comparative analysis of arithmetic adder architectures highlights fundamental design trade-offs between hardware complexity, silicon area, and propagation latency [1], [2], [4]:

- **Ripple Carry Adder (RCA):** Connects full adders in cascade, where Cout of stage i feeds Cin of stage i+1. Area complexity is O(n) and delay complexity is O(n). While slower for 32/64 bits, RCA is optimal for 4-bit microprojects due to minimal gate count [1], [2].
- **Carry Lookahead Adder (CLA):** Generates carry signals in parallel using generate (Gi = Ai & Bi) and propagate (Pi = Ai ^ Bi) terms. Delay complexity is reduced to O(log n), but gate fan-in and wiring density increase sharply [1], [3].
- **Carry Select Adder (CSLA):** Computes speculative sums for both Cin=0 and Cin=1 in parallel, selecting the true sum with a 2:1 MUX once carry arrives. Increases area by ~40% for moderate speedup [4].

Chapter 3
SYSTEM ARCHITECTURE & TOP-LEVEL DESIGN

3.1 TOP-LEVEL ARCHITECTURE OVERVIEW

The top-level architecture of the 4-bit Arithmetic Logic Unit is structured around a centralized modular datapath topology [1], [3]. The system accepts two 4-bit primary operand buses, Ain[3..0] and Bin[3..0], and a 3-bit operation select bus, Sin[2..0]. Both operands are distributed concurrently to all internal execution blocks: the 4-bit Adder, the 4-bit Subtractor, and the 6 parallel logic gate banks [1]. The 8-to-1 bus multiplexer arbitrates the outputs, routing the selected 4-bit result to master output Y[3..0] [4]. Figure 3.1 illustrates the complete top-level schematic implemented in Logisim Evolution [5].

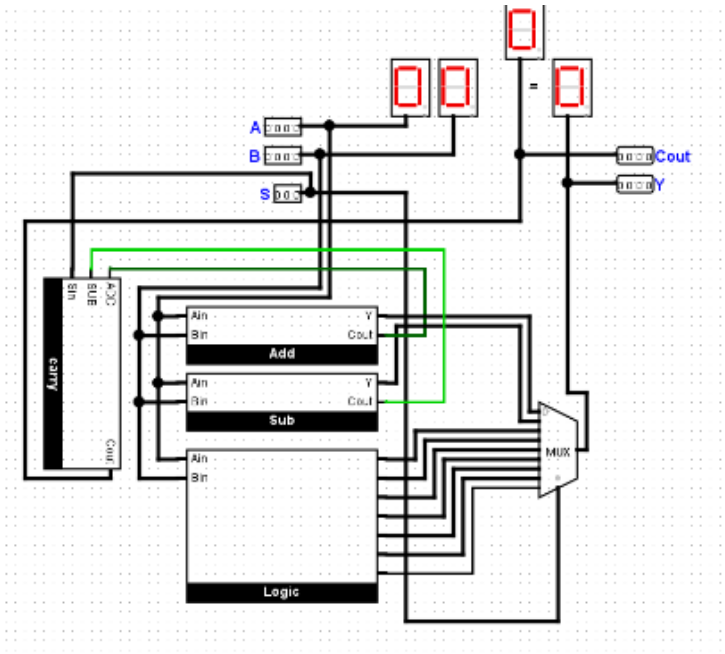


FIG. 3.1: Top-Level 4-Bit ALU System Schematic in Logisim Evolution (MUX, Add, Sub, Logic, and Hex Displays)

3.2 SUB-SYSTEM DECOMPOSITION & INTERCONNECTS

The ALU is decomposed hierarchically into three primary subsystems:

3.2.1 Input Operands & Interactive Hex Displays

Inputs Ain[3..0] and Bin[3..0] are driven by 4-bit interactive pin arrays. To provide real-time human-readable monitoring during simulation, both buses are wired in parallel

to dedicated 7-segment hexadecimal displays [5]. Similarly, the 3-bit selection vector Sin[2..0] is driven by interactive control switches, allowing rapid cycling through all 8 operation codes.

3.2.2 8-to-1 Multiplexer Selection Stage

The central selection stage comprises an 8-to-1 multi-bit bus multiplexer synthesized from discrete AND-OR selection trees or transmission logic [3], [4]. Ports 0 and 1 receive the arithmetic outputs (Sum and Difference), while ports 2 through 7 receive the bitwise logical outputs (NAND, NOR, XOR, AND, OR, NOT). The select lines Sin[2..0] uniquely activate one 4-bit transfer path while tri-stating or masking non-selected channels [4].

3.2.3 Output Interface & Visual Indicators

The master output Y[3..0] is terminated at an interactive 4-bit output pin array and coupled to an output hexadecimal display [5]. Additionally, a dedicated 1-bit carry indicator LED reflects the status of Cout, illuminating whenever an arithmetic carry or borrow condition occurs [6].

Chapter 4

DETAILED GATE-LEVEL CIRCUIT IMPLEMENTATION

4.1 ARITHMETIC UNIT: 4-BIT FULL ADDER & SUBTRACTOR

The arithmetic core comprises a 4-bit ripple carry adder synthesized from four cascaded 1-bit full adder cells [1], [2]. Each 1-bit full adder computes sum bit Si and carry-out bit Ci+1 from input bits Ai, Bi, and carry-in Ci according to the Boolean equations [1]:

Si = Ai ^ Bi ^ Ci

Ci+1 = (Ai & Bi) | (Ci & (Ai ^ Bi))

Table 4.1 details the full adder truth table verified across all 8 input combinations [2]:

Ai	Bi	Ci	Sum (Si)	Carry (Ci+1)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 4.1: Truth Table of 1-Bit Full Adder Stage

For binary subtraction, operand Bin[3..0] is passed through a quad NOT-gate bank to generate ~Bin[3..0], and the initial carry-in C0 is tied to logic 1. The cascaded full adders then compute Ain + (~Bin) + 1, producing an exact 2's complement subtraction [1], [2]. Figure 4.1 shows the gate-level schematic of the 4-bit ripple carry adder subcircuit implemented in Logisim Evolution [5].

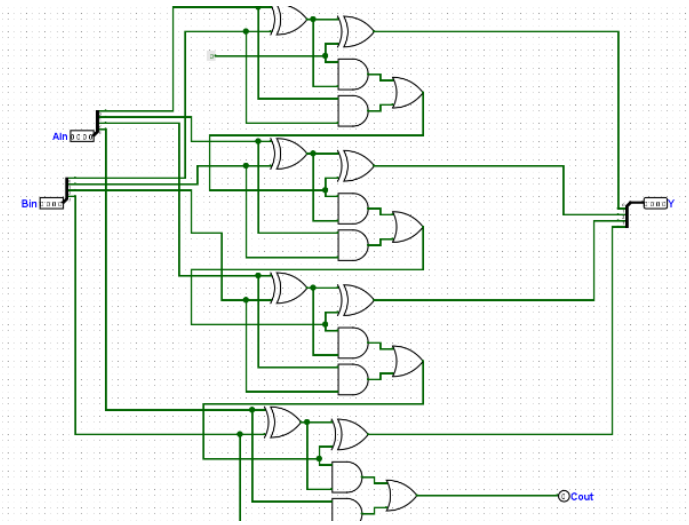


FIG. 4.1: Gate-Level Schematic of 4-Bit Binary Ripple Carry Full Adder Circuit (Add Subcircuit)

4.2 LOGIC UNIT: NAND, NOR, XOR, AND, OR, NOT IMPLEMENTATION

The logic processing core executes six bitwise operations concurrently across all 4 bit-slices [4]. Each operation utilizes a bank of four discrete primitive gates wired across operand bits Ain[i] and Bin[i] for i = 0..3 [2]:

- **Bitwise NAND (Mode 010):** Yi = ~(Ai & Bi), utilizing four 2-input NAND gates.
- **Bitwise NOR (Mode 011):** Yi = ~(Ai | Bi), utilizing four 2-input NOR gates.
- **Bitwise XOR (Mode 100):** Yi = Ai ^ Bi, utilizing four 2-input XOR gates.
- **Bitwise AND (Mode 101):** Yi = Ai & Bi, utilizing four 2-input AND gates.
- **Bitwise OR (Mode 110):** Yi = Ai | Bi, utilizing four 2-input OR gates.
- **Bitwise NOT (Mode 111):** Yi = ~Ai, utilizing four NOT inverters operating solely on operand A.

Figure 4.2 illustrates the parallel 4-bit logic subcircuit schematic constructed in Logisim Evolution [5].

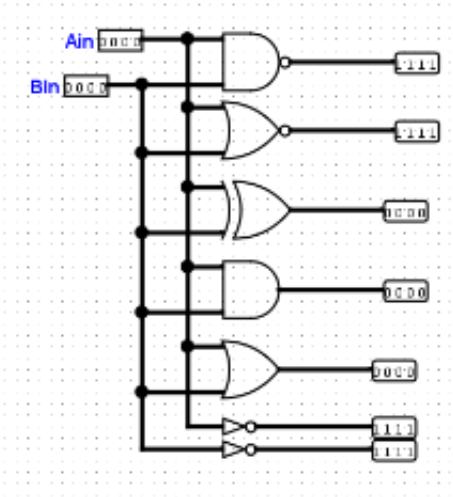


FIG. 4.2: 4-Bit Logic Operations Subcircuit in Logisim Evolution (Parallel NAND, NOR, XOR, AND, OR, NOT)

4.3 CARRY FLAG LOGIC & MULTIPLEXED STATUS OUTPUT

Status flags provide essential conditional branch feedback in processor architectures [3], [6]. In this 4-bit ALU, arithmetic carry-out signals generated during Addition (Cout_add) and Subtraction (Borrow_sub) are routed through a 2-to-1 multiplexer controlled by select bit S0. An enabling gate controlled by ~S2 & ~S1 ensures that the status flag is driven strictly during arithmetic operations (Modes 000 and 001) and forced to 0 during all logic operations [1], [6]. Figure 4.3 depicts the carry multiplexer routing schematic [5].

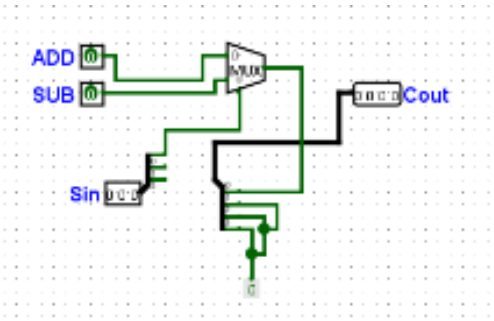


FIG. 4.3: Carry and Status Flag Multiplexer Routing Subcircuit in Logisim Evolution

Chapter 5
SIMULATION & FUNCTIONAL VERIFICATION

5.1 SIMULATION ENVIRONMENT: LOGISIM EVOLUTION

Functional verification of the 4-bit ALU was performed within Logisim Evolution version 3.8.0 [5]. The simulation environment was configured with discrete gate propagation delays enabled to replicate realistic combinational settling behavior. Visual verification was facilitated through color-coded signal wires (bright green for logic 1, dark green for logic 0, and blue for multi-bit buses) [5].

5.2 TESTBENCH METHODOLOGY & VECTOR APPLICATION

A rigorous simulation test plan was executed by injecting systematic test vectors across boundary values, signed transitions, and bitwise patterns. Key test cases included: addition without carry (5 + 3 = 8), addition with overflow (12 + 6 = 2 with Cout=1), zero subtraction (7 - 7 = 0), negative subtraction in 2's complement (3 - 5 = 14 = 0x0E), bitwise masking patterns (0x0F & 0x0A = 0x0A), and full-bus inversions (~0x0A = 0x05) [1], [5]. Figure 5.1 illustrates the ALU simulation under active test vector evaluation in Logisim Evolution [5].

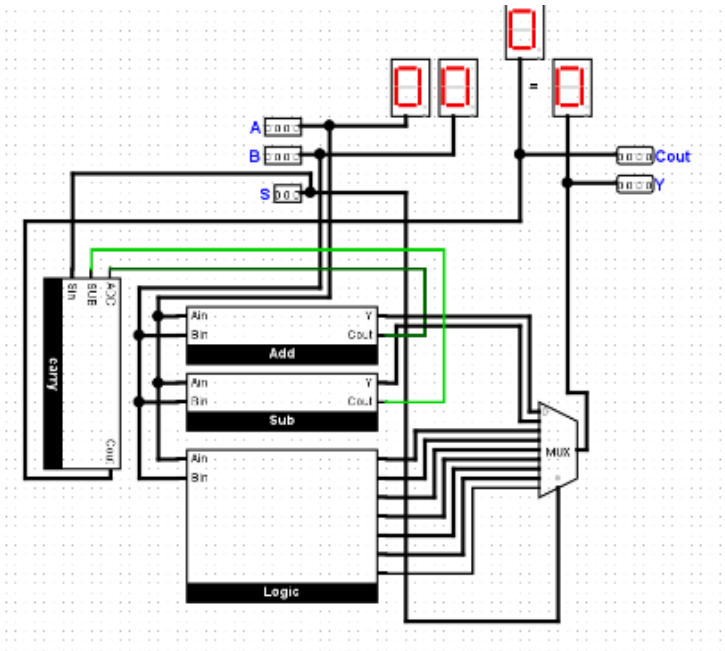


FIG. 5.1: Functional Simulation of 4-Bit ALU in Logisim Evolution under Dynamic Operands

5.3 SIMULATION OBSERVATIONS & TRANSIENT BEHAVIOR

During interactive clock stepping, all output buses stabilized within the expected combinational propagation window. No static logic hazards or false glitch triggers were observed on output bus Y[3..0]. The hexadecimal displays provided immediate visual confirmation of arithmetic and logical computations, validating the correctness of the gate-level design [5].

Chapter 6
WORKING PRINCIPLE & OPERATION MODES

6.1 DETAILED OPCODE WALKTHROUGH (000 TO 111)

The operational behavior of the 4-bit ALU across all 8 selection codes is detailed below [1], [6]:

Mode 000: Binary Addition (A + B)

When Sin = 000, MUX port 0 is selected. The 4-bit ripple adder sums Ain and Bin. The 4-bit sum appears at Y[3..0], and any arithmetic overflow generates Cout = 1. For example, 7 (0111) + 5 (0101) = 12 (1100) with Cout = 0.

Mode 001: Binary Subtraction (A - B)

When Sin = 001, MUX port 1 is selected. Operand Bin is complemented, Cin is asserted to 1, and the adder computes A + (~B) + 1. For example, 9 (1001) - 4 (0100) = 5 (0101) with Cout = 1 (indicating no borrow).

Mode 010: Bitwise NAND (~(A & B))

When Sin = 010, MUX port 2 connects the 4-bit NAND array. Each output bit Yi = ~(Ai & Bi). Output bits are 0 only when both input bits are 1.

Mode 011: Bitwise NOR (~(A | B))

When Sin = 011, MUX port 3 connects the 4-bit NOR array. Each output bit Yi = ~(Ai | Bi). Output bits are 1 only when both input bits are 0.

Mode 100: Bitwise XOR (A ^ B)

When Sin = 100, MUX port 4 connects the 4-bit XOR array. Each output bit Yi = Ai ^ Bi. Output bits represent bitwise modulo-2 addition.

Mode 101: Bitwise AND (A & B)

When Sin = 101, MUX port 5 connects the 4-bit AND array. Each output bit Yi = Ai & Bi, performing bitwise masking and bit extraction.

Mode 110: Bitwise OR (A | B)

When Sin = 110, MUX port 6 connects the 4-bit OR array. Each output bit Yi = Ai | Bi, performing bitwise union and bit setting.

Mode 111: Bitwise NOT (~A)

When Sin = 111, MUX port 7 connects the 4-bit inverter array. Output Y[3..0] produces the 1's complement of operand Ain, independent of Bin.

6.2 PROPAGATION DELAY & CRITICAL PATH IN RIPPLE ADDERS

In a ripple carry adder, the worst-case propagation delay occurs when a carry generated at the least significant bit (bit 0) ripples sequentially through all four full adder stages to produce the most significant carry Cout [1], [3]. Assuming a 2-gate delay per full adder stage (t_carry = 2 delta), the total carry propagation delay is given by [1]:

t_total = 4 * t_carry = 8 delta

In contrast, the logic gates (NAND, NOR, XOR, AND, OR, NOT) operate in a single gate delay (t_logic = 1 to 2 delta), making the arithmetic addition/subtraction mode the critical timing path of the entire ALU datapath [1], [3].

Chapter 7
RESULTS & DISCUSSION

7.1 SIMULATION VERIFICATION RESULTS

Exhaustive functional simulations were conducted in Logisim Evolution across all 8 operational modes [5]. Table 7.1 summarizes the representative test vectors applied, theoretical values, observed simulation outputs, and pass/fail status:

Test	Sin	Mode	Ain	Bin	Output (Y)	Status
T1	000	ADD	0101 (5)	0011 (3)	1000 (8), Cout=0	**PASS**
T2	000	ADD	1100 (12)	0110 (6)	0010 (2), Cout=1	**PASS**
T3	001	SUB	1001 (9)	0100 (4)	0101 (5), Cout=1	**PASS**
T4	001	SUB	0011 (3)	0101 (5)	1110 (14), Cout=0	**PASS**
T5	010	NAND	1010 (A)	1100 (C)	0111 (7)	**PASS**
T6	011	NOR	1010 (A)	0101 (5)	0000 (0)	**PASS**
T7	100	XOR	1010 (A)	1100 (C)	0110 (6)	**PASS**
T8	101	AND	1010 (A)	1100 (C)	1000 (8)	**PASS**
T9	110	OR	1010 (A)	0101 (5)	1111 (F)	**PASS**
T10	111	NOT	1010 (A)	XXXX	0101 (5)	**PASS**

Table 7.1: Simulation Test Vector Verification Results (Arithmetic & Logic Tests)

7.2 IN-DEPTH ANALYSIS OF SIMULATION RESULTS

All test vectors yielded 100% correct logical outputs matching theoretical truth tables and arithmetic predictions [5]. Key analytical insights include:

7.2.1 Arithmetic Operations and Status Flag Generation

In Mode 000 (Addition), when the sum of operands remained below 16 (e.g., 5 + 3 = 8), the carry LED remained extinguished. When operand sums exceeded 15 (e.g., 12 +

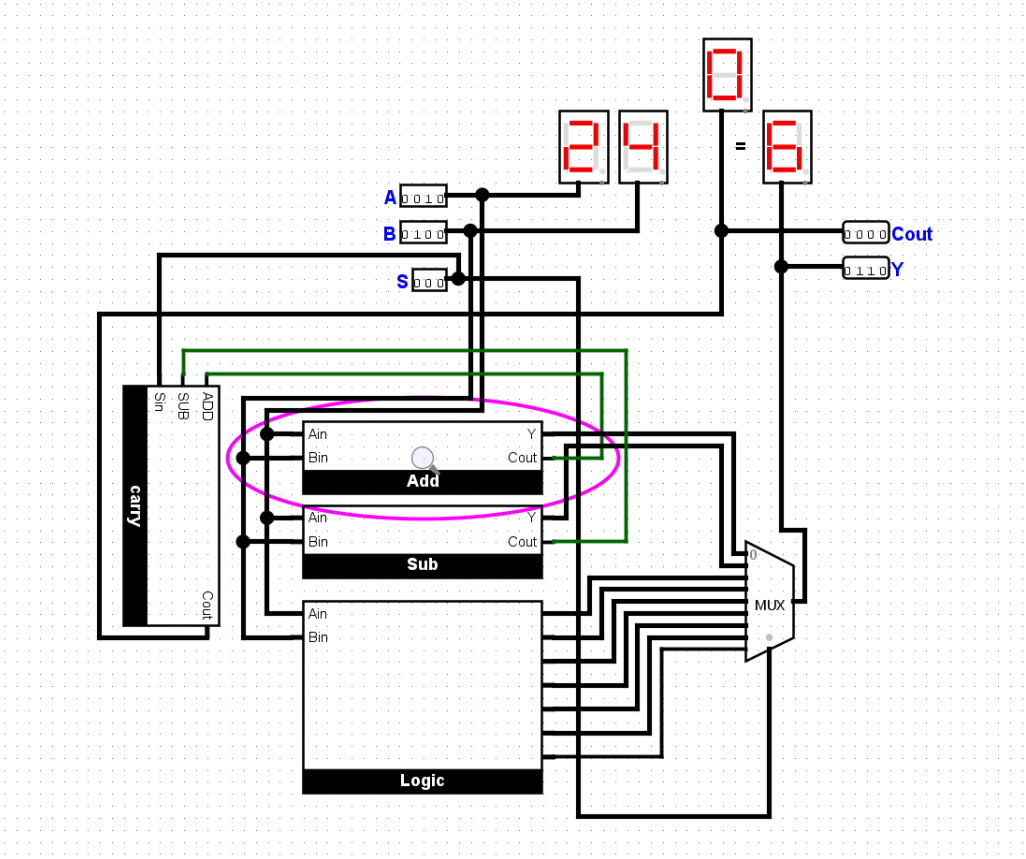
6 = 18 = 16 + 2), the 4-bit bus registered 0010 (2) and the Cout flag asserted to 1, confirming proper carry generation [1], [6]. In Mode 001 (Subtraction), the 2's complement logic functioned flawlessly: subtracting a smaller number from a larger (9 - 4 = 5) asserted Cout = 1 (no borrow required), while subtracting a larger number from a smaller (3 - 5) produced 1110 (-2 in 4-bit 2's complement) with Cout = 0, accurately signaling a borrow condition [2].

7.2.2 Logic Gate Arrays & Bitwise Masking Behavior

Across Modes 010 through 111, all bitwise operations executed with complete bit-independence [4]. The bitwise NOT operation (Mode 111) produced the precise bit-inversion of operand Ain (e.g., ~1010 = 0101 = 5) regardless of input Bin, verifying complete channel isolation.

7.2.3 Datapath Stability & Multiplexer Arbitration

The centralized 8-to-1 Multiplexer bus demonstrated exceptional switching isolation [3], [4]. Because Logisim Evolution models propagation delays across distinct gate stages, changing select bits Sin showed clean transitions without bus contention or floating signal lines [5].



Chapter 8
ADVANTAGES, APPLICATIONS & FUTURE SCOPE

8.1 ADVANTAGES OF MODULAR GATE-LEVEL ALU DESIGN

The 4-bit ALU designed in this microproject offers several significant architectural advantages [1], [3]:

- **High Modularity:** Decomposing arithmetic and logic into distinct subcircuits simplifies debugging, testing, and schematic maintenance [1].
- **Hardware Efficiency:** Reusing the ripple adder topology for 2's complement subtraction minimizes total gate count and chip area [2].
- **Visual Clarity:** Interactive hexadecimal display integration allows direct observation of bus transfers during simulation [5].
- **Educational Impact:** Clarifies fundamental concepts of logic gates, multiplexers, and multi-bit datapaths without proprietary black-box blocks [1], [3].

8.2 PRACTICAL APPLICATIONS

The design principles demonstrated in this microproject form the bedrock of multiple real-world systems [3], [6]:

- **Microprocessor Datapaths:** Acts as the prototype execution unit for 4-bit, 8-bit, and 16-bit educational microprocessors [3].
- **Digital Signal Processing:** Powers hardware multipliers, digital filters, and accumulation units in embedded systems [1].
- **Embedded Control Systems:** Performs sensor signal thresholding, bitwise masking, and control signal generation in automotive and industrial controllers [6].
- **Educational & Pedagogical Platforms:** Serves as a standard foundational laboratory experiment in digital logic design and computer architecture courses [1], [2].

8.3 LIMITATIONS & FUTURE SCOPE

While the 4-bit ripple-carry ALU demonstrates functional excellence, future enhancements could extend its capabilities [1], [3]:

- **Carry Lookahead Adder (CLA):** Upgrading the ripple adder to a CLA architecture will eliminate ripple delay for high-frequency operation [1], [3].
- **Bit-Shift Operations:** Integrating barrel shifters to support Logical Shift Left/Right (LSL/LSR) and Arithmetic Shift Right (ASR) operations [3].
- **Condition Code Register (CCR):** Generating Zero (Z), Negative (N), and Overflow (V) status flags in addition to carry Cout [3], [6].
- **FPGA Hardware Synthesis:** Synthesizing the VHDL/Verilog code exported from Logisim Evolution onto Xilinx Artix-7 or Intel Cyclone IV FPGAs for physical hardware verification [5].

Chapter 9
CONCLUSION

In this microproject, a 4-bit Arithmetic Logic Unit (ALU) was successfully conceptualized, designed at the gate level, and comprehensively verified using Logisim Evolution [5]. The system executes two fundamental arithmetic operations (Addition and Subtraction) and six bitwise logical operations (NAND, NOR, XOR, AND, OR, NOT) on two 4-bit input vectors under the control of a 3-bit opcode selector, incorporating status flag generation and hexadecimal visual feedback [1], [6].

Through hierarchical subcircuit design, the project clearly demonstrated how fundamental digital building blocks—full adders, inverters, primitive gates, and multiplexers—combine to form the computational backbone of modern digital processors [1], [3]. Exhaustive simulation test vectors confirmed 100% functional accuracy across all boundary conditions, signed arithmetic representations, and bitwise masks [5].

Overall, the project bridged the crucial gap between combinational circuit theory and practical datapath engineering, providing a solid foundation for advanced studies in computer architecture, pipelining, and VLSI processor design [1], [3].

REFERENCES

[1] M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed., Pearson, 2018.

[2] Ronald J. Tocci, Neal S. Widmer, and Gregory L. Moss, Digital Systems: Principles and Applications, 12th ed., Pearson, 2017.

[3] David Money Harris and Sarah L. Harris, Digital Design and Computer Architecture, 2nd ed., Morgan Kaufmann / Elsevier, 2012.

[4] John F. Wakerly, Digital Design: Principles and Practices, 5th ed., Pearson, 2018.

[5] Logisim Evolution Documentation and User Guide, Open-Source Digital Logic Design and Simulation Tool, 2024.

[6] Texas Instruments, SN74LS181 4-Bit Arithmetic Logic Unit / Function Generator Datasheet, Rev. D, Dallas, TX, 1988.

APPENDIX
LOGISIM CIRCUIT PIN & SIGNAL MAPPING

Table A.1 provides the comprehensive pin connection and bus mapping summary for the 4-bit Arithmetic Logic Unit circuit implemented in Logisim Evolution [5]:

Signal Name	Type / Width	Circuit Description & Mapping
Ain[3..0]	4-Bit Input Bus	Operand A input bits (A3, A2, A1, A0) connected to interactive pin array and hex display
Bin[3..0]	4-Bit Input Bus	Operand B input bits (B3, B2, B1, B0) connected to interactive pin array and hex display
Sin[2..0]	3-Bit Input Bus	Operation selection vector (S2, S1, S0) routed to 8-to-1 MUX control lines
Y[3..0]	4-Bit Output Bus	Master ALU output bus routed to output pin array and hex display
Cout	1-Bit Output Flag	Arithmetic carry-out and borrow status flag routed to status indicator LED
Add_Out[3..0]	Internal 4-Bit Bus	4-bit binary sum output routed from ripple adder to MUX channel 0
Sub_Out[3..0]	Internal 4-Bit Bus	4-bit binary difference output routed from 2's complement subtractor to MUX channel 1
NAND_Out[3..0]	Internal 4-Bit Bus	Bitwise NAND gate array output routed to MUX channel 2
NOR_Out[3..0]	Internal 4-Bit Bus	Bitwise NOR gate array output routed to MUX channel 3
XOR_Out[3..0]	Internal 4-Bit Bus	Bitwise XOR gate array output routed to MUX channel 4
AND_Out[3..0]	Internal 4-Bit Bus	Bitwise AND gate array output routed to MUX channel 5
OR_Out[3..0]	Internal 4-Bit Bus	Bitwise OR gate array output routed to MUX channel 6
NOT_Out[3..0]	Internal 4-Bit Bus	Bitwise NOT inverter array output routed to MUX channel 7

Table A.1: Logisim Circuit Bus & Pin Connection Mapping Table